

main repository. He can easily do so because no other change has been committed to the main repository by other developers. The difference between his local repo and the main repo is two commits (E and F); D is still the head commit in the main repo, so by applying E and F, he can make the main repo exactly the same as his local repo. This type of merging, without any conflicts, applying newly added commits to the parent repo, is known as fast-forward merge.

Now, let's look at a conflicting merge version of the above scenario: by the time Dev-1 made changes, some other developer (Dev-2) added some other changes to the main repo. The main repo now looks like what follows:

A-B-C-D-G

Now, when Dev-1 tries to merge, Git responds that a fast-forward merge cannot be done. Then Dev-1 has to rebase his code on top of the main repo, and resubmit to apply the patches, i.e., the developer has to fetch the code from the main branch, and apply his local commits on top of the commit G. This is fairly simple. Git provides easy methods to resolve conflicts. When Dev-1 tries to merge, it will say that the merge failed and ask him to manually resolve conflicts to continue—and will list the files in which conflicts exist. The developer can open the files and see that Git has inserted markers like '<<<', which separate code from the local branch and code from the other repo. You should then choose the correct lines of code that you need, make the necessary modifications to resolve the issues, delete the '<<<' markers and then continue the merge operation. The operation will succeed and the developer can sync the code to the main repo. Merges and merge conflicts can occur between different repositories as well as branches.

Basic usage

Let us learn the basic usage commands and flows for Git.

Setting up the identity

Initially, after installing Git, you need to set up the identity for the committer. You can either set a global identity for user-wide configuration, or create a repository-specific identity. For example, to set a global user identity, use the following commands:

```
$ git config --global user.name "Sarath Lakshman"
$ git config --global user.email "sarathlakshman@slynx.org"
```

If you want to set a repository-specific private configuration, change the working directory to a Git repository and invoke the above commands without the `--global` option.

You can list all the Git configurations with (repository) *git*

config list, or (global) *git config --global list*. You can check the current user name and email with *git config user.name* and *git config user.email*.

Creating an empty repository

To create a repository in the current directory, use *git init*.

Adding files to the staging area

Every time you make a change to a file in the repository, you have to add that file to the staging area before making a commit. Use *git add file1 file2* to add the file(s). It is also possible to select code chunks to be added from a file using the `-p` (prompt) option with *add*: *git add -p filename*. You can select the code chunk using the *y/n* prompt.

Committing changes from the staging area

When you run *git commit*, it asks for a commit message. A nice commit message has a one-line subject with a length of 65 characters, and a multi-line description of 70 characters, separated by a blank line. The *Git commit* command takes the following options:

- *git commit -m "Message"*
- *git commit -a* (all modified files in the current directory are moved to the staging area and added to the commit)
- *git commit file1 file2* (If you want to move *file1* and *file2* to the staging area and create the commit in one shot, files can be specified with *git commit*.)

If you created a commit, and think that the message was not right, or some more code should be added to the commit, you can modify the commit using `--amend`. To interactively modify the commit, step by step, use *git commit --amend -i*.

Looking at the Git log

Commit logs are very important for version control systems.

To view the commit logs, use the following commands:

```
$ git log
$ git log -2 (show last two commits)
$ git log -p
```

The `-p` option stands for *pretty*. It also displays the differences between the last commits.

```
$ git log --since 'May 22 2012'
$ git log --before "2 days"
$ git log --after "3 days"
```

It is also possible to list the log related to a commit by giving the commit ID (hash) as the parameter to the *Git log* command (*git log commit_id*).

Status of the repository

The *git status* command is very important. It is used to display the current status of the repository, and will show the contents